Document number: P0479R0
Date: 2016-10-16
Audience: Evolution Working Group, SG14
Reply-to: Clay Trychta <clay.trychta@gmail.com>

Attributes for Likely and Unlikely Branches

I. Table of Contents

II. Introduction

Two new attributes `[[likely]]` and `[[unlikely]]` are proposed. These attributes will serve as hints on the likelihood that a subsequent branch is taken. Compilers may use these hints to improve the code they generate in various ways.

III. Motivation and Scope

Compiler's optimizers often have no information relating to branch probability which can lead to suboptimal code generation. In many cases the excellent dynamic branch predictors on modern processors can make up for this lack of information. However, in some cases code may execute more slowly than necessary even though the programmer knew the probability of particular branches being executed because they did not have an easy way to communicate this to the compiler.

Several existing compilers including GCC and Clang implement `__builtin_expect` which can be used to communicate branch probability information to the optimizer [1] [2]. Looking at usages of `__builtin_expect` in the wild it seems that by far most common usage is through defining a `likely` macro (`#define likely(x) __builtin_expect(!!(x), 1)`) and an `unlikely` macro (`#define unlikely(x) __builtin_expect(!!(x), 0)`). Probably the largest project that makes use of this functionality is the linux kernel which uses `likely` over 3,000 times and `unlikely` over 14,000 times [3]. Other large projects using this functionality include Mozilla with over 200 usages of `MOZ_LIKELY` and over 7,000 usages of `MOZ_UNLIKELY` and Chromium which has hundreds of instances of `LIKELY` and `UNLIKELY` [4] [5].

Some of the possible code generation improvements from using branch probability hints include:

- Multiple microarchitectures should be able to benefit from reordering basic blocks and branches to improve instruction cache utilization and keep the most common paths in the pipeline [6]. These improvements can sometimes be significant. For example, on x86, macro-fusion/branch fusion can allow for multiple extra instructions to be executed in a single cycle [7] [8] [9].
- It is common for compilers to generate predicated instructions (eg. cmov) likely due to their generally good performance characteristics and resiliency against unpredictable conditions. Intel recommends against using these instructions for predictable branches due to the overhead of executing both paths of a branch [10]. Testing done while writing this proposal did find better performance when using compare/jump instead of cmov for highly predictable branches.
- Some microarchitectures, such as Power, have branch hints which can override dynamic branch prediction [11].
- AMD recommends structuring code for its processors so that branches are not taken which consumes fewer branch prediction resources and does not incur the prediction-based bubble of a correctly-predicted taken branch [12].

Since this proposal has made claims about the potential to improve generated code and performance I have created a few small examples showing the possible benefits. All testing was performed with the GCC 6.1.1 20160621 compiler and run on a Xeon E3 1245 v3. All examples were compiled with `-O3 -march=haswell` plus defines to control behavior and some other test specific flags mentioned below. Tests were run three times using `perf stat` and the median value was taken.

This first example examines clamping integer values into the interval [0, 65535]. The code was compiled without hints and with hints that the values would not need to be adjusted. The compiled example was then run with various percentages of values that needed to be adjusted. The below results show that the hinting was able to improve performance when most values were in the expected range, but as more values were outside of the expected range the code compiled with the branch hints became slower then the code compiled without any hints.

% values outside [0, 65535] (approx)	Time (secs) no hint	Time (secs) `unlikely`
.1%	8.01	6.17 (-1.84)
1%	8.25	6.76 (-1.49)
5%	9.36	7.91 (-1.45)
10%	10.83	9.42 (-1.41)
50%	22.55	23.42 (+0.87)
90%	33.91	35.42 (+1.51)
95%	34.35	35.88 (+1.53)
99%	34.48	36.02 (+1.54)

99.9%	34.53	36.03 (+1.5)

The next example examines the tradeoffs between using conditional move and compare/jump. This time the code was compiled without hints, with a hint to expect the comparison to succeed, and with a hint to expect the comparison to fail. The compiled example was then run with various percentages of values for which the comparison would succeed. I was unable to get GCC to stop generating conditional move instructions using only `__builtin_expect` so I compiled with `-fno-if-conversion -fno-tree-loop-if-convert` to simulate it. Recent versions of Clang (3.9+) change from generating conditional moves to comparison and jumps based on the usage of `__builtin_expect` [13].

The results show that if almost all values (99%+) will take a particular branch then generating a comparison and branch is better. When the data is slightly more random, generating a conditional move can be much better than branching code. For the 50% case I saw that over 20% of branches were mispredicted for the cases using branch instructions.

% values set (approx)	Time (secs) cmov	Time (secs) cmp/jmp `unlikely`	Time (secs) cmp/jmp `likely`
.1%	4.98	3.47 (-1.51)	5.58 (+0.6)
.5%	4.96	3.76 (-1.2)	5.95 (+0.99)
1%	4.97	4.12 (-0.85)	6.31 (+1.34)
50%	4.91	36.02 (+31.11)	33.74 (+28.83)
99%	4.95	5.44 (+0.49)	5.04 (+0.09)
99.5%	4.91	5.02 (+0.11)	4.56 (-0.35)
99.9%	4.92	4.73 (-0.19)	4.09 (-0.83)

Finally, the impact of `__builtin_expect` was measured on two open source projects since their code should be a bit closer to what would be seen in actual codebases than the previous examples.

The first test used pugixml to parse an example XMark file from memory.

The subsequent tests ran the bench.c benchmark from picohttpparser. These tests were run with and without `-march=haswell`.

All of the tests were run without using `__builtin_expect`, using `__builtin_expect` with the opposite values (1 for unlikely and 0 for likely), and using `__builtin_expect` with the proper values (0 for unlikely and 1 for likely).

These tests show that proper usage of `__builtin_expect` can have a positive impact on runtime performance. For the cases where the measured times are relatively close together I was able to run each version repeatedly and saw very similar numbers to what is reported below so the differences do seem to be real.

Case	No expect	Opposite expect	Regular expect
pugixml	126.0936 ms	128.6235 ms (+2.5299)	122.7716 ms (-3.322)
picohttpparser -march=haswell	1.7857 s	1.7763 s (-0.0094)	1.7119 s (-0.0738)
picohttpparser	3.5989 s	3.7150 s (+0.1161)	3.0369 s (-0.562)

The preceding examples show that allowing the programmer to specify branch hints can lead to improved code generation and decreased run time. Due to these potential benefits this proposal aims to standardize the common likely/unlikely usage pattern so it can be used in a portable manner across compilers without macros. It also hopes to serve as encouragement for more compilers to implement this functionality.

IV. Impact On the Standard

This proposal has minimal impact on the current standard as it proposes two new attributes which do not change program semantics.

V. Design Decisions

Related Use Cases

From discussions regarding this proposal it seems that there are several related cases that people would like to be able to provide hints to the compiler for:

- Annotating branches that are very likely or unlikely to be true.
- Indicating whether or not a branch is predictable. This is useful in cases where it is known ahead of time that the values being branched on are going to follow a predictable pattern, but a particular direction does not have a high likelihood (in which case likely/unlikely could be used).
- Annotating the priority of a particular branch direction. This is orthogonal to the likelihood of a branch being taken and indicates that the a particular direction should be optimized for even if it is unlikley in order to decrease latency for that particular case (even at the cost of slowing down the more common case).

This proposal currently focuses on solving the first of the above cases as support for such annotations already exist in at least two compilers and their usage is relatively widespread. The other cases either have limited (eg. `__builtin_unpredictable` in Clang) or no support in existing compilers which makes it more difficult to judge the benefits of any particular approach to solving those cases [14].

Possible Objections

Objection #1: Can this feature easily result in code pessimization?

> Yes, as shown by the conditional move example misusing a branch hint can definitely result in worse code with a much longer run time. This feature is primarily meant to be used after a hotspot has been found in existing code. The proposed attributes are not meant to be added to code without measuring the impact to ensure they do not end up degrading performance.

Objection #2: Why should branch hints be standardized when many compilers implement PGO?

> Branch hints are definitely not meant to replace PGO; if PGO can be used it should be. PGO may be able to give the optimizer even more information then a branch hint. However, PGO and branch hints do not need to be mutually exclusive; branch hints could act as another indicator of expected frequent usage of particular branches.
>
> Also, PGO is not always easy to use. PGO necessitates creating a realistic test scenario to profile which, depending on the application, may be difficult and opens up the possibility that some important real world usage may be missed. The additional build tooling to keep the profile up to date, particularly across multiple OS's and compilers may also be challenging.
>
> Additionally, certain kinds of applications, such as those which may be distributed to clients who then build and run their own plugins in the distributed application may be very difficult or impossible to generate a realistic benchmark for, as there is no way of knowing how the client will use the application. Similar problems could also occur if an application is highly configurable; creating test scenarios and builds for all different possible configurations may not be realistic.
>
> Finally, PGO sometimes does the exact opposite of what is wanted for certain low-latency use cases. PGO tries to make the common case faster, but it may actually be more important to make the uncommon cases faster so that when they do occur they run as quickly as possible.

VI. Technical Specifications

In section 6 (stmt.stmt) paragraph 1, change the grammar to allow an optional attribute in conditions consisting of an expression.

> *condition:*
> *attribute-specifier-seq*$_{opt}$ *expression*

Add the `likely` attribute.

7.6.N Likely attribute [dcl.attr.likely]

The *attribute-token* `likely` indicates that an expression has a high probability of being true. It shall appear at most once in each *attribute-list* and no *attribute-argument-clause* shall be present.

The attribute may be applied to the condition of an `if` statement.

[*Note:* Implementations are encouraged to use the `likely` attribute as a hint for optimization purposes — *end note*]

[*Example:*

```
if ([[likely]] foo()) {
  do_something();
}
```

Implementations are encouraged to optimize for the case where foo() returns true. — *end example*]

Add the `unlikely` attribute.

7.6.N Unlikely attribute [dcl.attr.unlikely]

The *attribute-token* `unlikely` indicates that an expression has a high probability of being false. It shall appear at most once in each *attribute-list* and no *attribute-argument-clause* shall be present.

The attribute may be applied to the condition of an `if` statement.

[*Note:* Implementations are encouraged to use the `unlikely` attribute as a hint for optimization purposes — *end note*]

[*Example:*

```
if ([[unlikely]] foo()) {
  do_something();
}
```

Implementations are encouraged to optimize for the case where foo() returns false. — *end example*]

VII. Acknowledgements

Thank you to Bartosz Bielecki, Carl Cook, Matt Dziubinski, Mathias Gaunard, Paul Hampson, and others in SG14 for sharing their insights and suggestions.

VIII. References

1. https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html
2. http://llvm.org/docs/BranchWeightMetadata.html#built-in-expect-instructions
3. https://groups.google.com/a/isocpp.org/d/msg/sg14/ohFcWdlvrh0/FJXC94MeAgAJ
4. https://dxr.mozilla.org/mozilla-central/search?q=MOZ_UNLIKELY&redirect=false
5. https://cs.chromium.org/search/?q=LIKELY+case:yes+exact:yes&sq=package:chromium&type=cs
6. http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-optimization-manual.pdf, page 575
7. http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-optimization-manual.pdf, page 108
8. http://support.amd.com/TechDocs/47414_15h_sw_opt_guide.pdf, page 122
9. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=67153#c23
10. http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-optimization-manual.pdf, page 101
11. https://www.power.org/wp-content/uploads/2013/05/PowerISA_V2.07_PUBLIC.pdf, page 64
12. http://support.amd.com/TechDocs/47414_15h_sw_opt_guide.pdf, page 129
13. https://reviews.llvm.org/D19488
14. http://clang.llvm.org/docs/LanguageExtensions.html#builtin-unpredictable

Appendix A

```
#include <array>
#include <cstdint>
#include <random>

#ifdef EXPECT_VAL
#define expect(x) __builtin_expect(!!(x), EXPECT_VAL)
#else
#define expect(x) (x)
#endif

std::uint16_t clamp(int i) {
  if (expect(i < 0)) {
    return 0;
  } else if (expect(i > 0xFFFF)) {
    return 0xFFFFu;
  }

  return i;
}

int main() {
  std::mt19937 gen(42);
  std::bernoulli_distribution d(.001), up_down(.5);

  std::array<int, 1'000'000> data;
  for (std::size_t i = 0; i != data.size(); ++i) {
    if (d(gen)) {
        data[i] = up_down(gen) ? -1 : 0xFFFFF;
    } else {
        data[i] = 1;
    }
  }

  std::uint32_t result = 0;
  for (int i = 0; i != 10000; ++i) {
    for (auto val : data) {
      result += clamp(val);
    }
  }

  return result > 5 ? 1 : 2;
```

```
}
```

Appendix B

```
#include <array>
#include <cstdint>
#include <random>

#ifdef EXPECT_VAL
#define expect(x) __builtin_expect(!!(x), EXPECT_VAL)
#else
#define expect(x) (x)
#endif

int main() {
  std::mt19937 gen(42);
  std::bernoulli_distribution d(.001);
  std::array<std::uint32_t, 1'000'000> data;
  for (std::size_t i = 0; i != data.size(); ++i) {
    data[i] = d(gen) ? 1 : 0;
  }

  std::uint32_t result = 0;
  for (int i = 0; i != 10000; ++i) {
    for (auto val : data) {
      if (expect(val != 0)) {
        result = i;
      }
    }
  }

  return result > 5 ? 1 : 2;
}
```